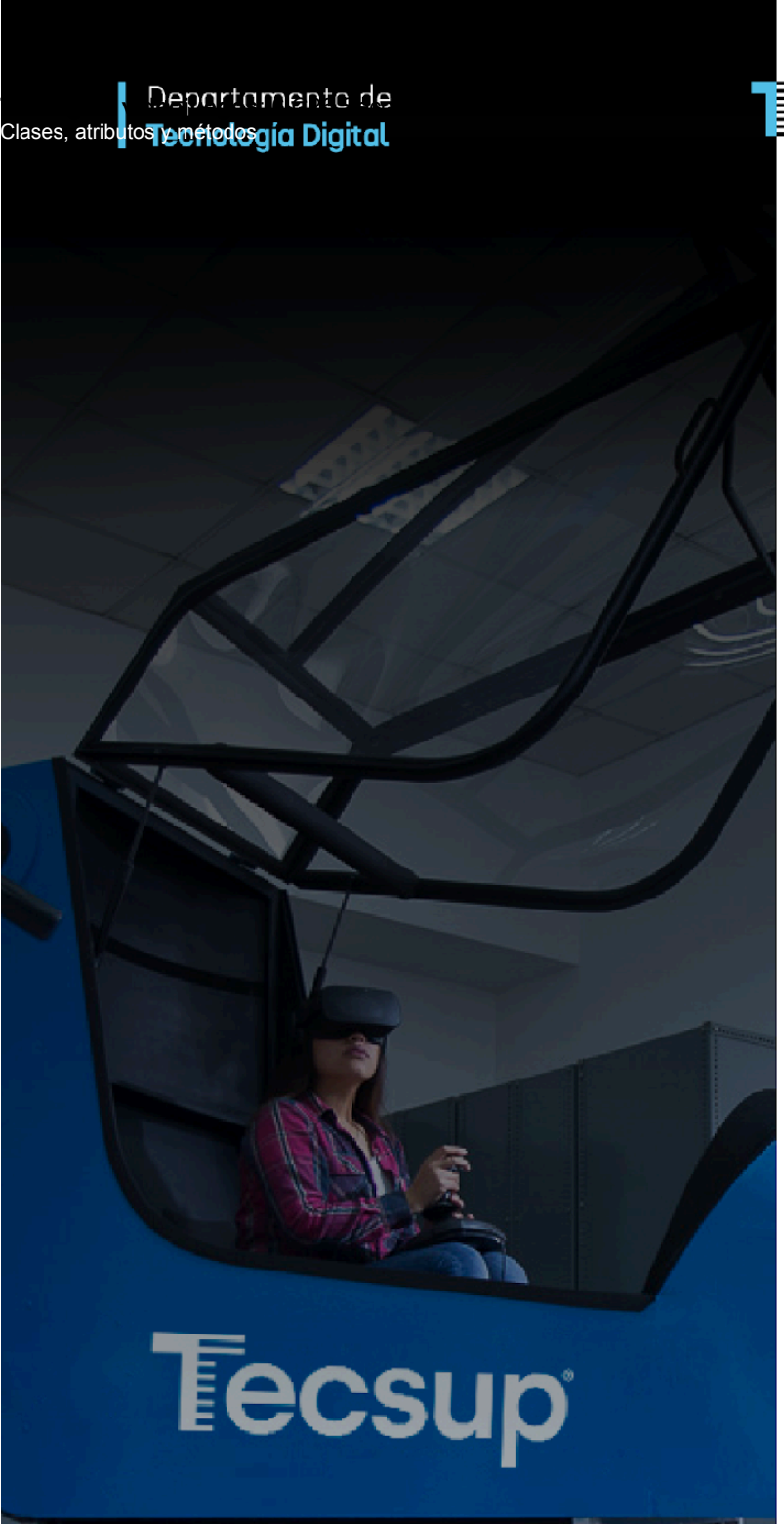
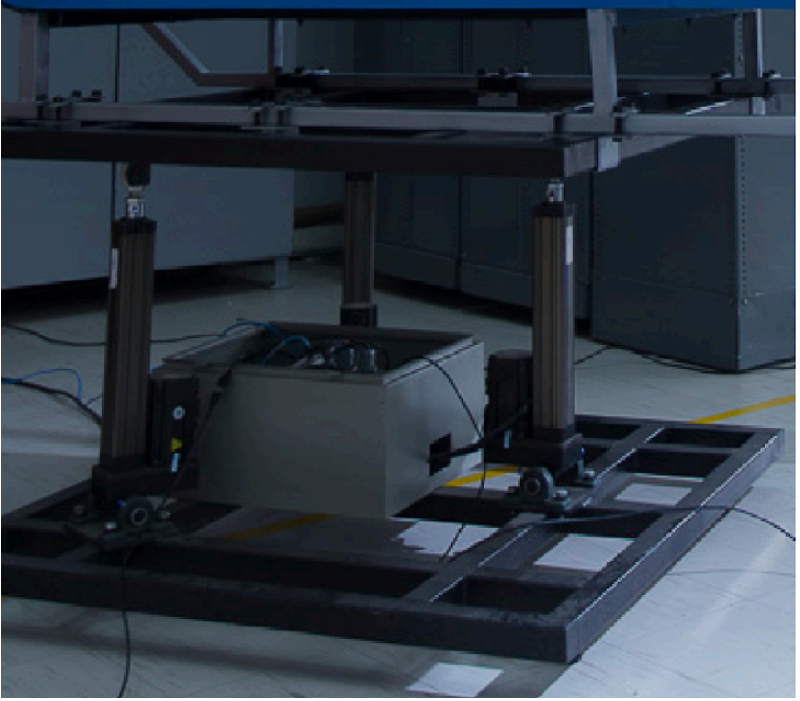
Departamento de Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO





LABORATORIO N° 06

Desarrollo de Simuladores y Videojuegos

DOCENTE:

Bestard Aroche, Yunior

CURSO:

Testing de Videojuegos
6 - C26 - Sección A

TESTING DE VIDEOJUEGOS: Sistema de Trabajo de Testing 1

I. Capacidades

- Comprender el concepto de sistema de trabajo de testing y sus componentes.
- Analizar estrategias modernas de testing y vincularlas a proyectos de tesis.
- Elaborar casos de prueba iniciales aplicados a la solución propuesta en la tesis.
- Organización de tareas con metodologías agiles.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 11.
- Los softwares requeridos ya se encuentran instalados en los laboratorios.

VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual

VII. Procedimiento

1) Listado de actividades planificadas

Elabora una lista de **4–8 actividades**. Para cada una completa los campos:

ACTIVIDADES PLANIFICADAS – APLICACIÓN A LA TESIS

Actividad 1 – Validación del estado de vida del jugador

Objetivo: comprobar que la variable de vida del jugador mantenga una condición válida durante la ejecución.

Entregable: script de validación y evidencia de ejecución.

Criterios de aceptación: la validación funciona con el valor esperado y detecta cuando la condición deja de cumplirse.

Responsable(s): Ismael Santiago Carrasco Huamán.

Evidencia: captura de Unity con la condición correcta y captura con la condición incorrecta.

Actividad 2 – Validación de objetos críticos del escenario

Objetivo: comprobar que un objeto necesario para el funcionamiento del videojuego permanezca activo.

Entregable: script de validación y evidencia de ejecución.

Criterios de aceptación: el objeto permanece activo en la condición correcta y la validación detecta cuando es desactivado.

Responsable(s): Ismael Santiago Carrasco Huamán.

Evidencia: capturas del Inspector y de la ejecución.

Actividad 3 – Validación de posición del jugador/objeto

Objetivo: comprobar que un objeto se encuentre aproximadamente en la posición establecida.

Entregable: script de validación y evidencia de ejecución.

Criterios de aceptación: la posición correcta pasa la validación y una posición modificada es detectada.

Responsable(s): Ismael Santiago Carrasco Huamán.

Evidencia: capturas de la posición correcta y de la posición modificada.

Actividad 4 – Validación del Tag del jugador

Objetivo: comprobar que el GameObject utilizado como jugador tenga el Tag esperado.

Entregable: script de validación y evidencia de ejecución.

Criterios de aceptación: el Tag Player pasa la validación y un Tag diferente provoca el fallo esperado.

Responsable(s): Ismael Santiago Carrasco Huamán.

Evidencia: capturas del Inspector y de la ejecución.

- **Nombre**
- **Objetivo** (qué resuelve)
- **Entregables** (qué se entrega: documento, escena, prototipo, dataset, etc.)
- **Criterios de aceptación** (condiciones observables/medibles)
- **Responsable(s)**
- **Evidencias** (qué subirá el equipo para demostrar avance)

EJERCICIO 1 – DESARROLLO

Assert seleccionado: Assert.AreNotEqual.

Script completado:

```
using UnityEngine;
using UnityEngine.Assertions;
public class HealthAssertion : MonoBehaviour
{
   public int health = 100;
   void Update()
   {
      Assert.AreNotEqual(0, health);
   }
}
```

Explicación: se utiliza Assert.AreNotEqual porque la condición esperada es que la vida del personaje no sea igual a cero durante la ejecución.

Evidencia correcta: [INSERTAR CAPTURA – health = 100].

Evidencia incorrecta: [INSERTAR CAPTURA – health = 0 y la validación detectando la condición].

Error que puede detectar: una vida que llegue a cero cuando el sistema todavía espera que el jugador esté activo podría indicar un problema en la lógica de vida.

2) Implementación de las actividades planificadas

Selecciona **4-8 actividades impleméntalas**. Para **cada actividad** completa y publique por imágenes o video la evidencia.

3) Elabore los siguientes ejercicios

Ejercicio 1 — Validación del sistema de vida

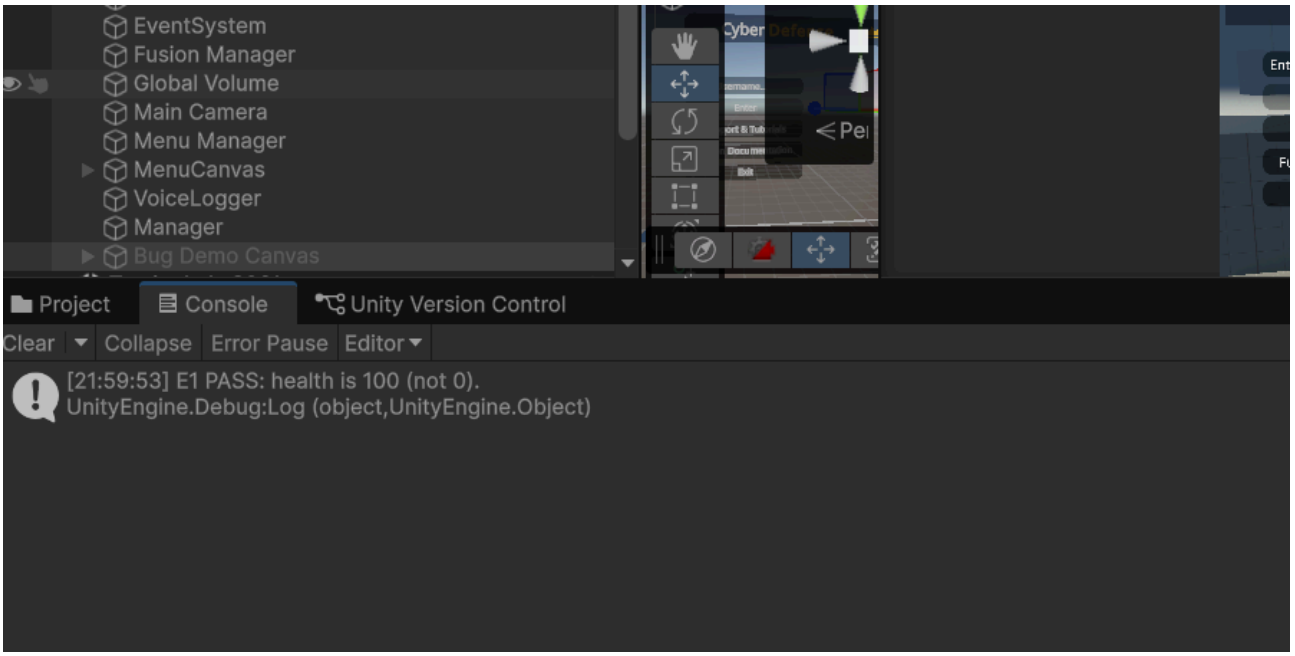
Objetivo: comprobar durante la ejecución que la vida del personaje respete la condición indicada.

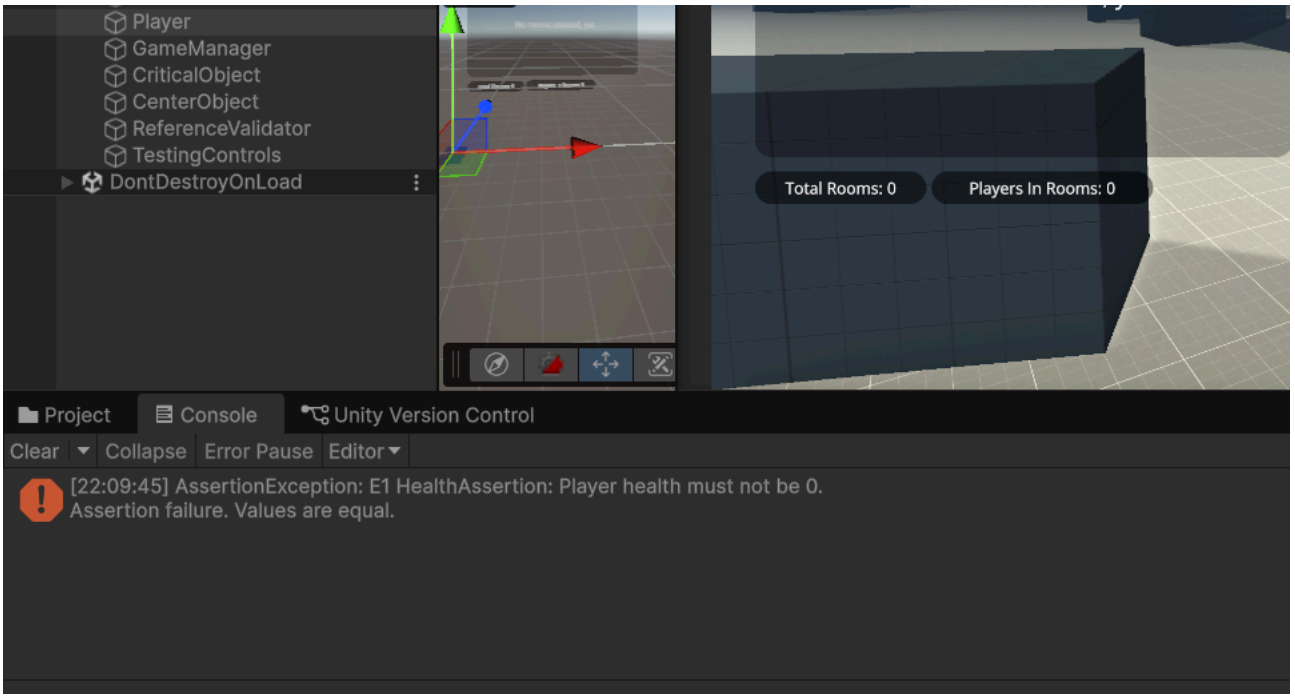
Crea el script:

```
using UnityEngine;
using UnityEngine.Assertions;

public class HealthAssertion : MonoBehaviour
{
```

public int health = 100;





EJERCICIO 2 – DESARROLLO

Assert seleccionado: Assert.IsTrue.

Script completado:

```
using UnityEngine;

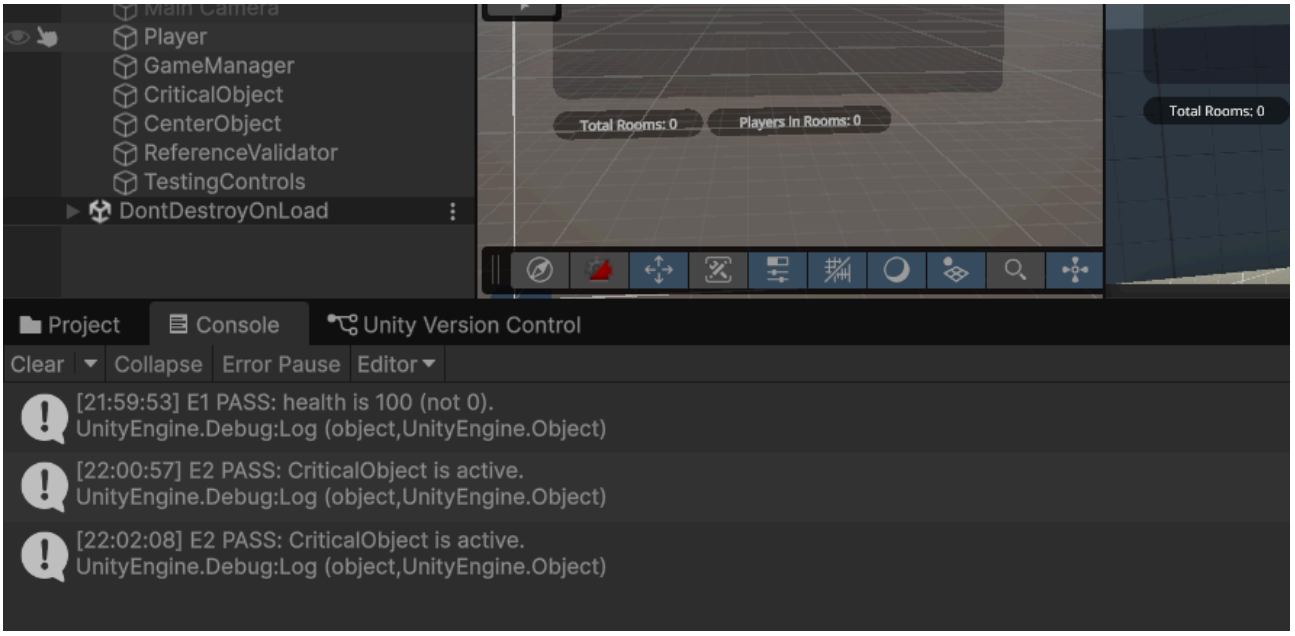
using UnityEngine.Assertions;

public class ActiveObjectAssertion : MonoBehaviour

{
```

```
    public GameObject target;
    void Update()
    {
        Assert.IsTrue(target.activeSelf);
    }
}
```

Explicación: se utiliza Assert.IsTrue porque la condición esperada es verdadera cuando el GameObject se encuentra activo. La propiedad activeSelf permite conocer si el propio GameObject está activado.



Error que puede detectar: un objeto crítico desactivado puede impedir el funcionamiento de una parte necesaria del videojuego.

```
    void Update()
    {
        // Agregue aquí la validación correspondiente.
    }
}
```

Indicaciones

1. Crea un GameObject llamado Player.
2. Agrega el script HealthAssertion.
3. Configura inicialmente:

health = 100

4. Ejecuta el juego.
5. Durante la ejecución cambia el valor de health a:

0

6. Implementa dentro de Update() el tipo de Assert que permita detectar esta condición.
7. Explica por qué seleccionaste ese método.
8. Registra evidencia de la condición correcta y de la condición incorrecta.

Ejercicio 2 — Validación de un GameObject activo

Objetivo: comprobar que un GameObject necesario para el funcionamiento del videojuego permanezca activo.

Crea:

```
using UnityEngine;
using UnityEngine.Assertions;


public class ActiveObjectAssertion : MonoBehaviour
{
   public GameObject target;


   void Update()
   {
      // Agregue aquí la validación correspondiente.
   }
}
```

Indicaciones

1. Crea un GameObject llamado:

GameManager

2. Crea otro llamado:

CriticalObject

3. Agrega ActiveObjectAssertion al GameManager.
4. Desde el Inspector asigna CriticalObject a target.
5. Ejecuta la escena.
6. Comprueba inicialmente que CriticalObject esté activo.

7. Durante Play Mode desactiva CriticalObject.
8. Selecciona e implementa el Assert que permita verificar que el objeto debe permanecer activo.
9. Explica qué propiedad utilizaste para conocer el estado del GameObject.

Ejercicio 3 — Validación de posición

Objetivo: comprobar que un GameObject permanezca aproximadamente en una posición determinada.

Crea:

using UnityEngine;

using UnityEngine.Assertions;

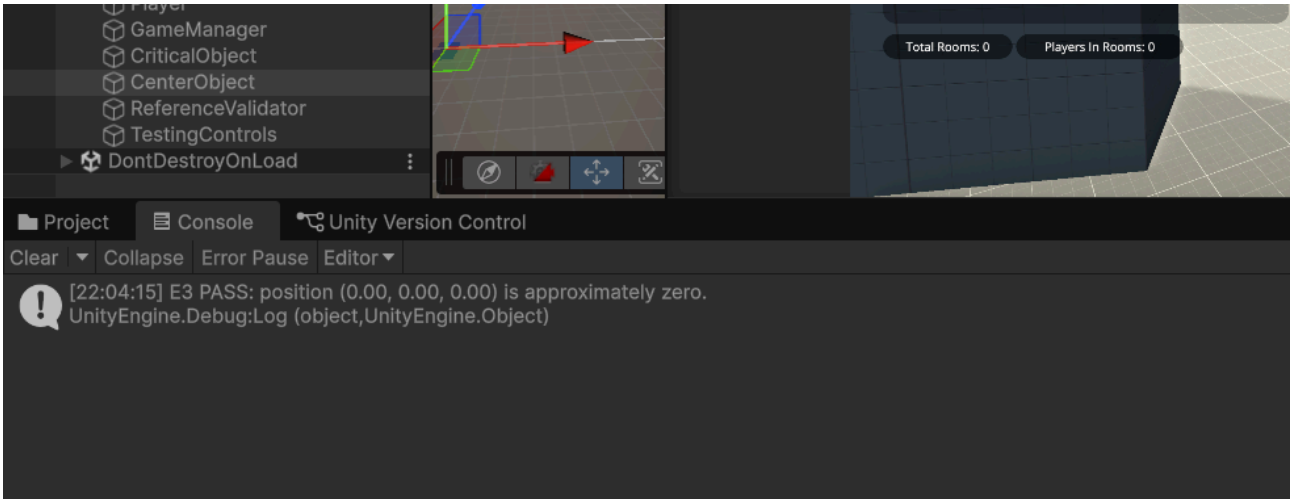
EJERCICIO 3 – DESARROLLO

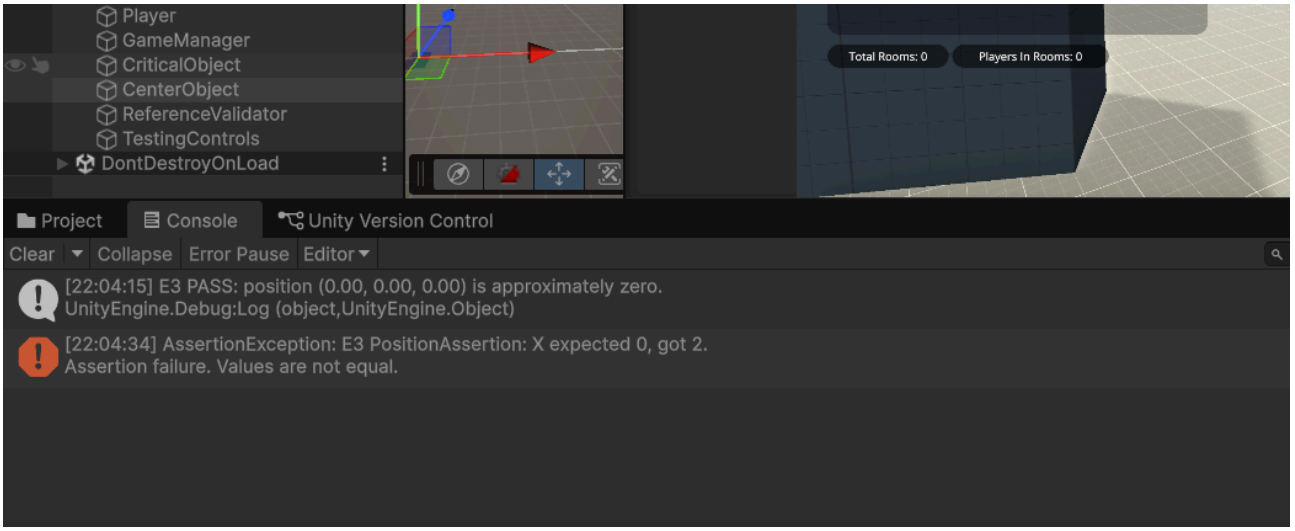
Assert seleccionado: Assert.AreApproximatelyEqual.

Script completado:

```
using UnityEngine;
using UnityEngine.Assertions;
public class PositionAssertion : MonoBehaviour
{
   void Update()
   {
      float positionX = transform.position.x;
      float positionY = transform.position.y;
      float positionZ = transform.position.z;
      Assert.AreApproximatelyEqual(0f, positionX);
      Assert.AreApproximatelyEqual(0f, positionY);
      Assert.AreApproximatelyEqual(0f, positionZ);
   }
}
```

Explicación: se utiliza Assert.AreApproximatelyEqual porque las coordenadas son valores float y la comparación aproximada permite validar que el objeto se encuentre suficientemente cerca de la posición esperada.





Error que puede detectar: una posición incorrecta de un objeto importante podría provocar que aparezca fuera de su ubicación esperada dentro de la escena.

```
public class PositionAssertion : MonoBehaviour
{
    void Update()
    {
        float positionX = transform.position.x;
        float positionY = transform.position.y;
        float positionZ = transform.position.z;

        // Agregue aquí las validaciones necesarias.
    }
}
```

Indicaciones

1. Crea un GameObject llamado:

   CenterObject
2. Configura su posición inicial:

   X = 0

   Y = 0

   Z = 0
3. Agrega PositionAssertion.
4. Ejecuta la escena.
5. Implementa las validaciones necesarias para comprobar que el objeto se encuentre aproximadamente en el origen.
6. Durante la ejecución mueve el objeto, por ejemplo:

   X = 2

   Y = 0

   Z = 0
7. Observa el comportamiento.
8. Explica por qué para valores float puede ser más conveniente realizar una comparación aproximada que una comparación exacta.

El PPT trabaja precisamente la comparación aproximada para coordenadas de transform.position. PPT-06-YBESTARD-2026-02

Ejercicio 4 — Validación del Tag del jugador

Objetivo: comprobar que un GameObject tenga el Tag esperado.

Crea:

```
using UnityEngine;

using UnityEngine.Assertions;


public class PlayerTagAssertion : MonoBehaviour

{
```

El laboratorio permitió aplicar validaciones mediante Assert para comprobar condiciones que deben cumplirse durante la ejecución de un videojuego.

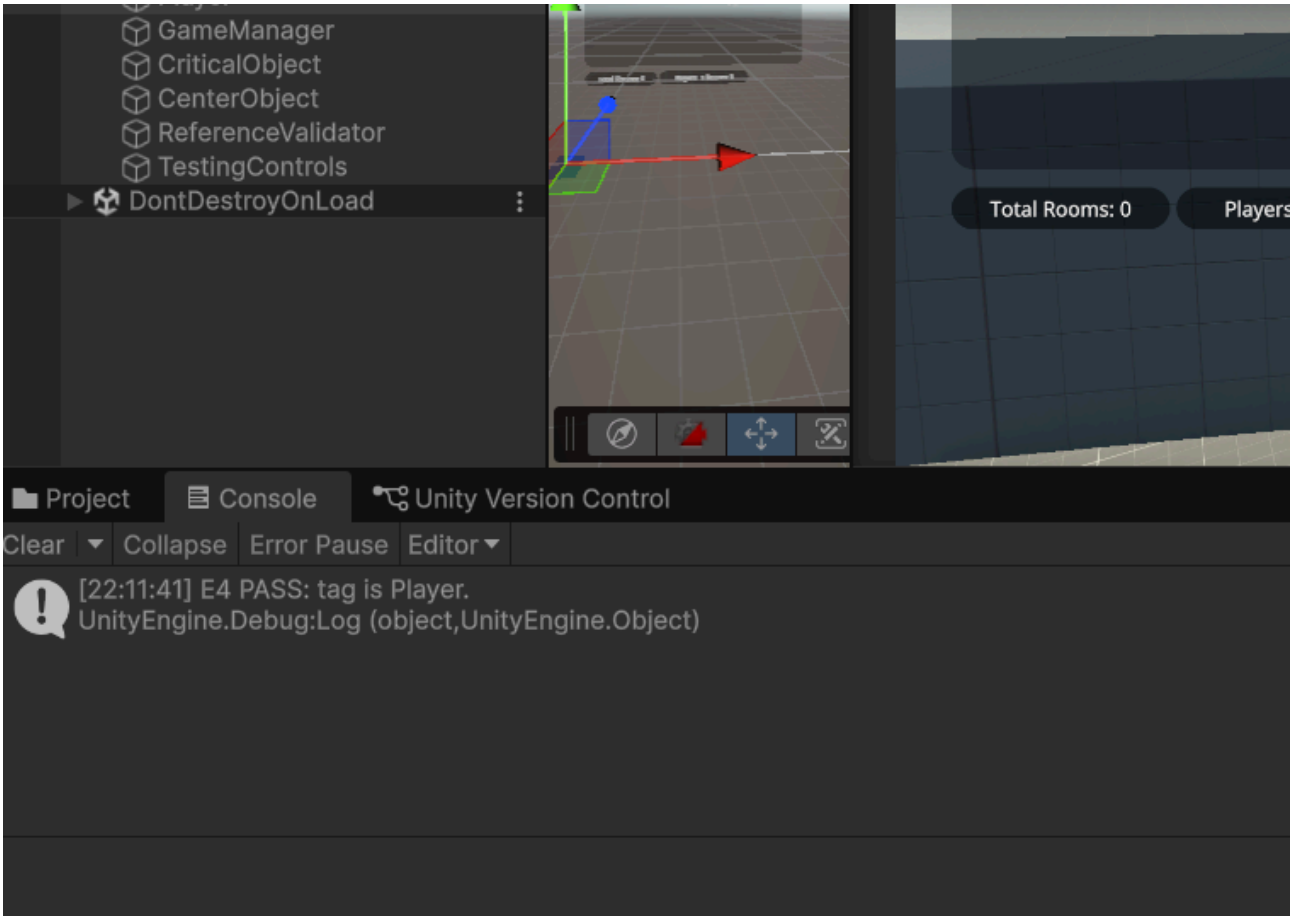
Las pruebas permiten detectar intencionalmente estados incorrectos y comprobar que la validación reaccione ante ellos.

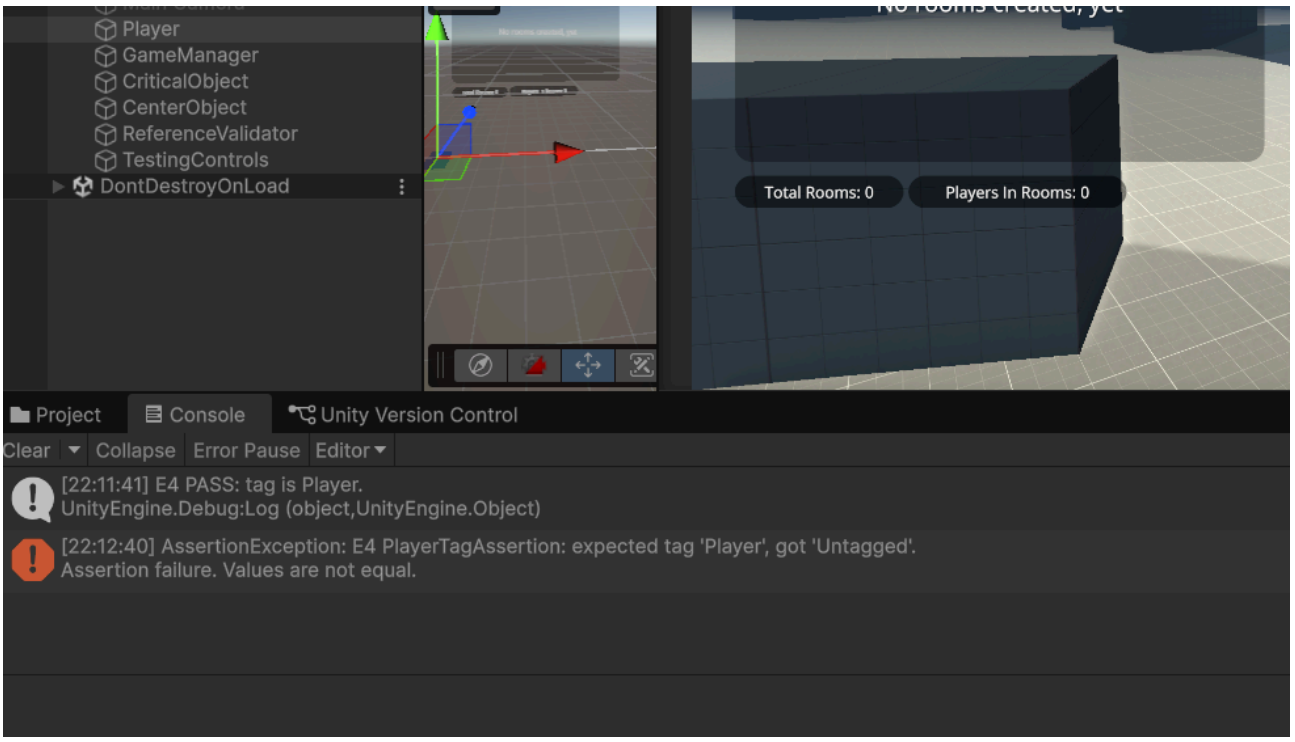
Estas validaciones pueden adaptarse posteriormente a elementos de la tesis, como jugadores, objetos críticos, posiciones y referencias necesarias para el funcionamiento del videojuego.

```
    void Update()
```

```
    {
        string currentTag = gameObject.tag;

        Assert.AreEqual("Player", currentTag);
    }
}
```





Indicaciones

1. Utiliza el GameObject del jugador.
2. Configura inicialmente su Tag como:

Player

3. Agrega PlayerTagAssertion.
4. Implementa una validación que permita comprobar que el Tag actual sea el esperado.
5. Ejecuta la escena.
6. Después cambia temporalmente el Tag a:

Untagged

7. Ejecuta nuevamente.
8. Explica:
    - cuál era el valor esperado;
    - cuál era el valor obtenido;
    - qué método de Assert seleccionaste.

El ejemplo original del PPT comprueba que el Tag sea "Player". PPT-06-YBESTARD-2026-02

Ejercicio 5 — Validación de una referencia

Objetivo: comprobar el estado de una referencia durante la ejecución.

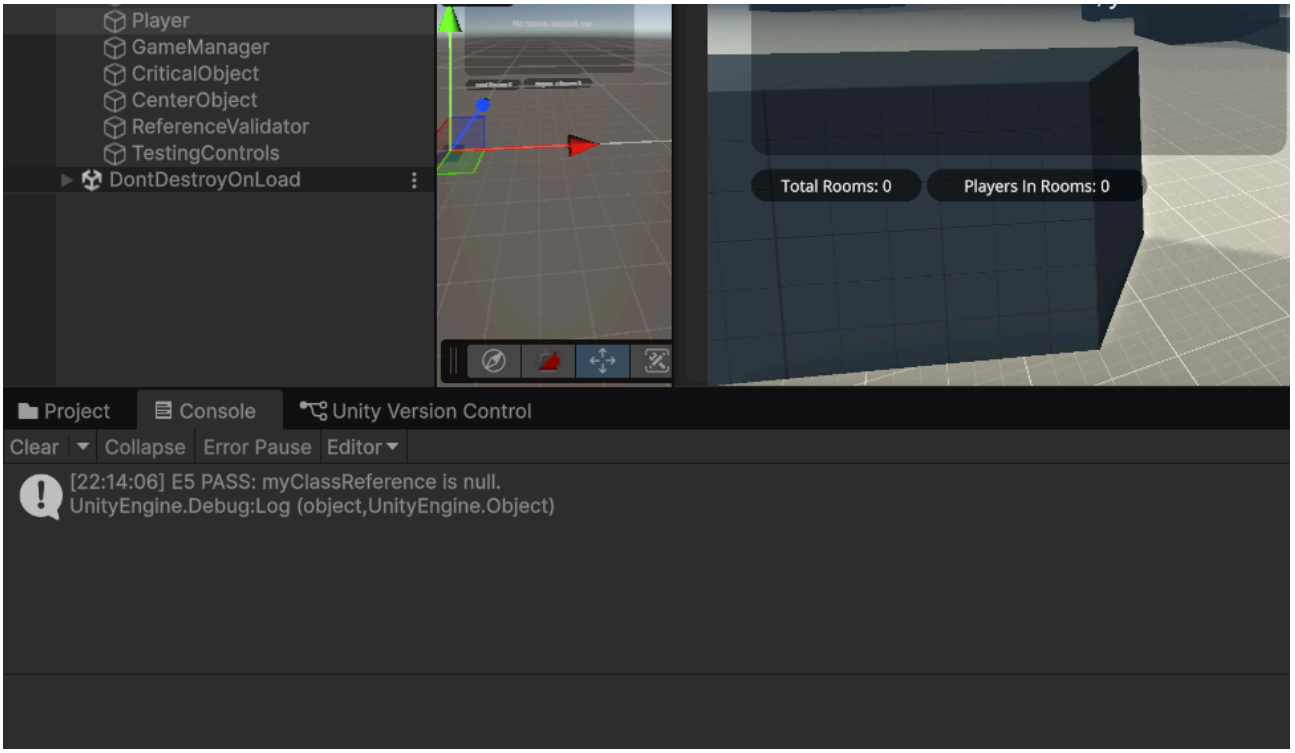
Crea:

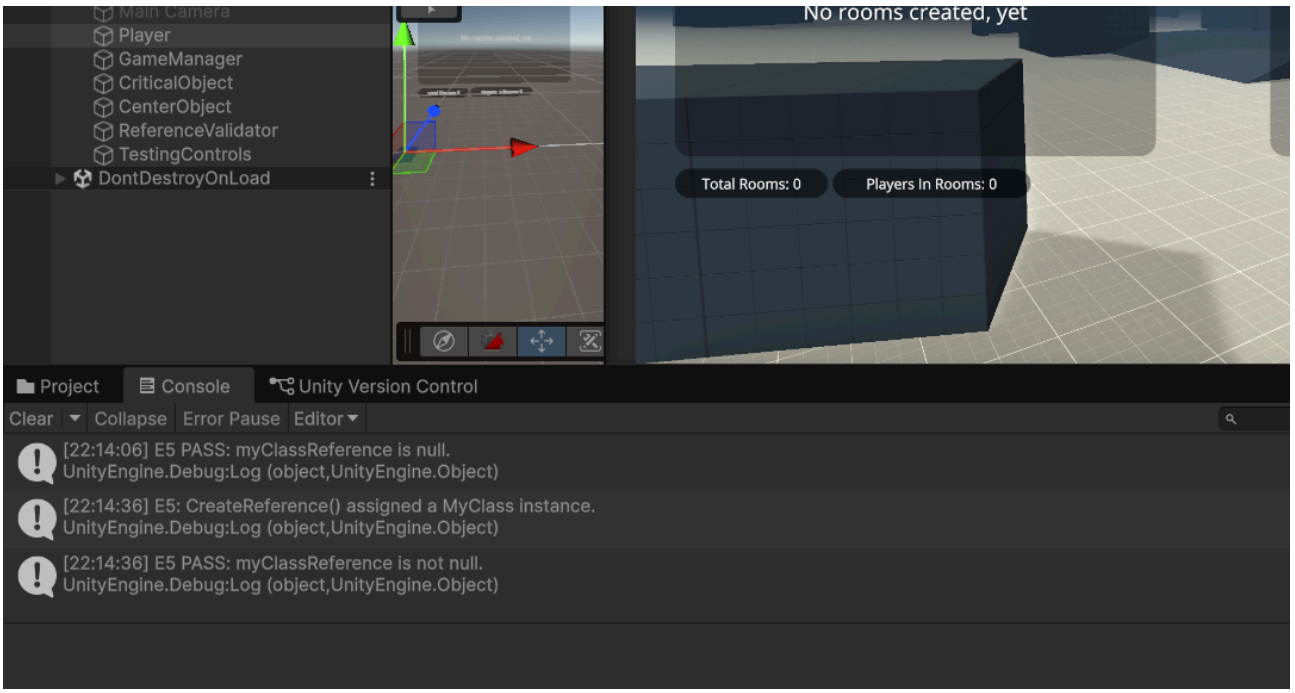
```
using UnityEngine;
using UnityEngine.Assertions;

public class ReferenceAssertion : MonoBehaviour
{
    private MyClass myClassReference;
      private bool initialCheckDone;
    void Update()
    {
      if (!initialCheckDone) { Assert.IsNull(myClassReference); initialCheckDone = true; }
    }

    public void CreateReference()
    {
        myClassReference = new MyClass(); Assert.IsNotNull(myClassReference);
    }
}

public class MyClass
{
}
```





Indicaciones

1. Crea un GameObject llamado:

ReferenceValidator

2. Agrega ReferenceAssertion.
3. Al iniciar la escena:

myClassReference

no contiene todavía una instancia.
4. Selecciona el Assert adecuado para comprobar el estado esperado de la referencia.
5. Después agrega temporalmente:

```
void Start()
{
    CreateReference();
}
```

6. Ejecuta nuevamente.
7. Observa qué ocurre cuando la referencia cambia.
8. Explica por qué el Assert elegido es apropiado para este caso.

El PPT incluye un caso equivalente utilizando una referencia de clase dentro de un MonoBehaviour. PPT-06-YBESTARD-2026-02

Lo que debe resolver el estudiante

En ninguno de los cinco ejercicios debes proporcionarles la línea de Assert. Ellos deben determinar cuál corresponde entre los métodos trabajados en clase, por ejemplo:

AreEqual

AreNotEqual

IsTrue

IsFalse

IsNull

IsNotNull

AreApproximatelyEqual

Para cada ejercicio deben entregar:

1. El script completado.
2. El Assert que seleccionaron.
3. Una explicación de por qué eligieron ese Assert.
4. Evidencia de una ejecución donde la condición sea correcta.
5. Evidencia de una ejecución donde provoquen intencionalmente que la condición deje de cumplirse.
6. Una breve explicación de qué error del videojuego podría detectar esa validación.

Conclusiones:

NOTA PARA LA ENTREGA: Las evidencias pueden presentarse mediante capturas de pantalla. Reemplazar cada texto [INSERTAR CAPTURA ...] por la imagen correspondiente antes de entregar.